



CS-317

Operating Systems

LECTURE 16



Concurrency

BOOK 1 CHAPTER 5



In the last lecture...

- ▶ We explored the implementation of counting semaphores.
- ▶ We were introduced to one of the classical problems of synchronization, the producer / consumer problem and its versions.
- ▶ We implemented code for the producer / consumer problem using binary and counting semaphores.
- ▶ We were introduced to the concept of messages.



Message Passing

CONCEPT, USE AND IMPLEMENTATION



Addressing

- ▶ It is necessary to specify the destination of a message in the send primitive.
- ▶ Most implementations allow the receiving process to specify the source from which it wants to receive a message.
- ▶ The various schemes for specifying processes in send and receive primitives fall into two categories:
 - ▶ Direct addressing, and
 - ▶ Indirect addressing.



Direct addressing

- ▶ The **send primitive** includes a specific **identifier** of the **destination process**.
- ▶ In the receive primitive:
 - ▶ A process can specify a source process if it knows the **ID of the sending process ahead of time**, or
 - ▶ A process may use **implicit addressing**: the source parameter of the receive primitive contains return value of the receive operation.



Indirect addressing

- ▶ Messages are not sent directly from sender to receiver but rather are sent to a shared data structure consisting of queues that can temporarily hold messages.
- ▶ Such queues are generally referred to as mailboxes.
- ▶ For two processes to communicate, one process sends a message to the appropriate mailbox and the other process picks up the message from the mailbox.
- ▶ By decoupling the sender and receiver, indirect addressing allows for greater flexibility in the use of messages.

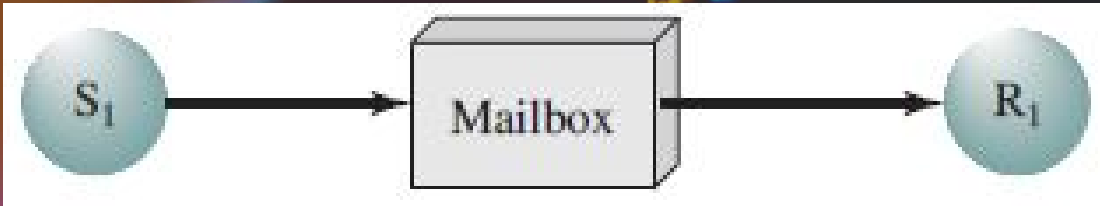


Sender-receiver relationships

- ▶ The relationship between senders and receivers can be:
 - ▶ One to one,
 - ▶ Many to one,
 - ▶ One to many, or
 - ▶ Many to many.



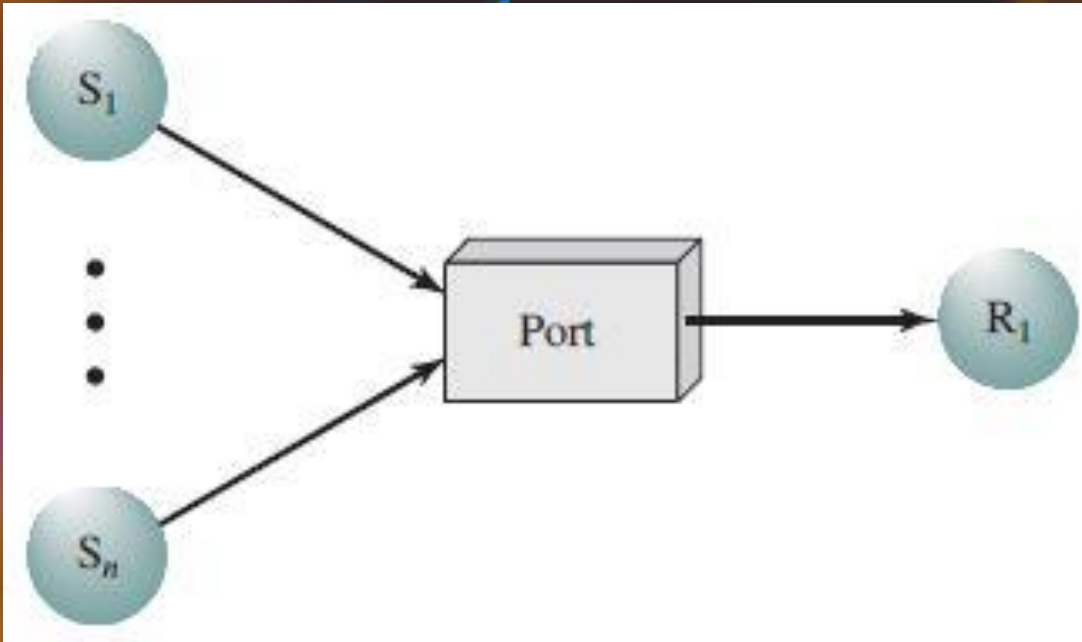
One-to-one relationship



- ▶ Only **one sender** sends to a mailbox.
- ▶ Only **one receiver** receives from the mailbox.
- ▶ A one-to-one relationship allows a **private communications** link to be set up between two processes.
- ▶ **This insulates their interaction from erroneous interference from other processes.**



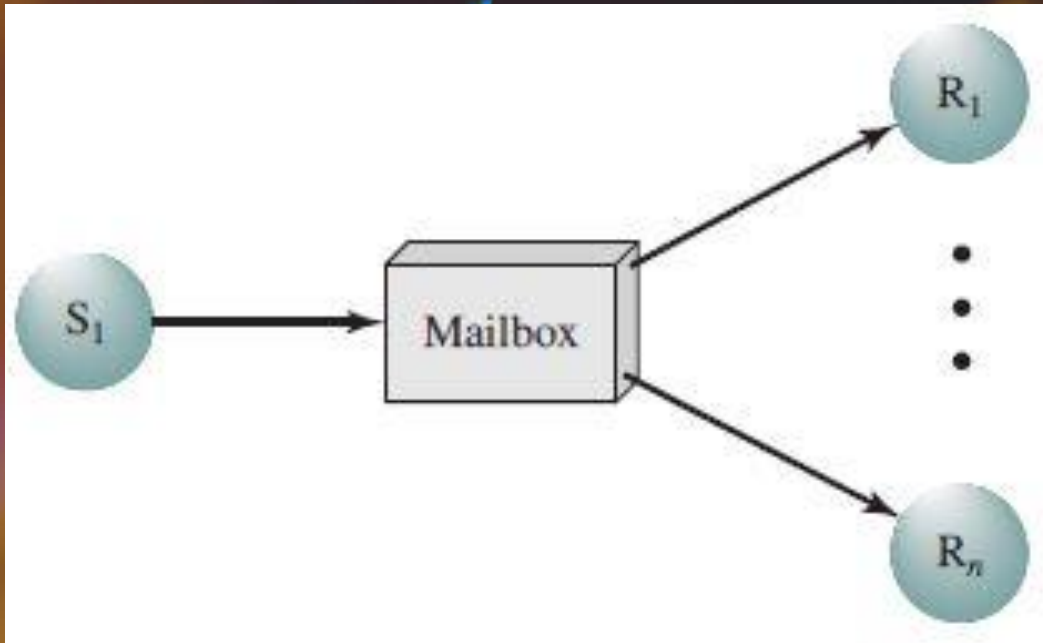
Many-to-one relationship



- ▶ Many senders send to a mailbox.
- ▶ One receiver receives from the mailbox.
- ▶ A many-to-one relationship is useful for client/server interaction; one process provides service to several other processes.
- ▶ In this case, the mailbox is often referred to as a port.



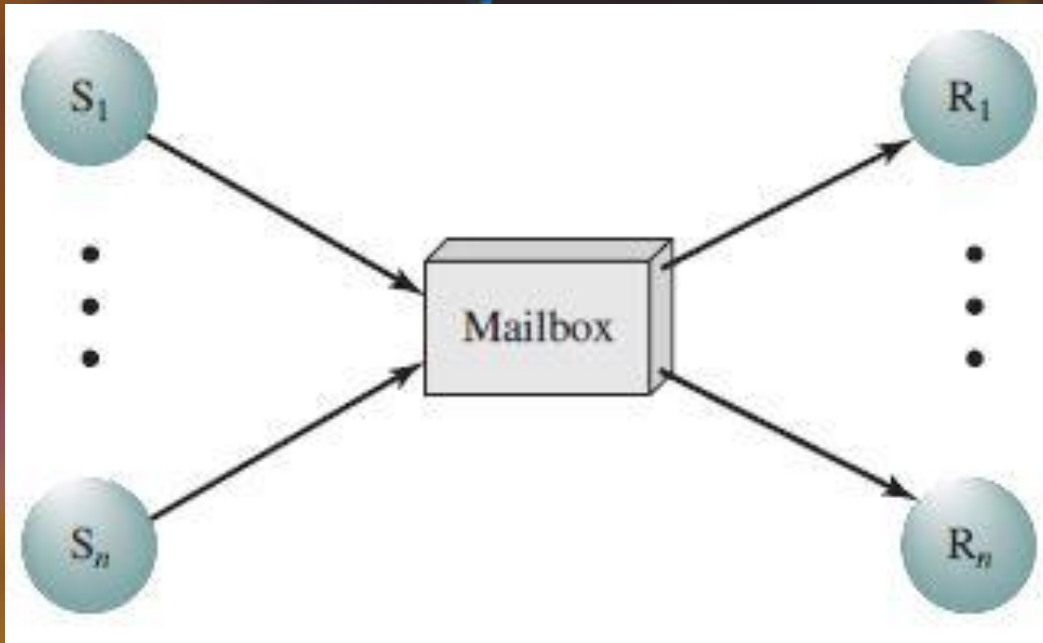
One-to-many relationship



- ▶ One sender sends to a mailbox.
- ▶ Many receivers receive from the mailbox.
- ▶ A one-to-many relationship allows for one sender and multiple receivers.
- ▶ It is useful for applications where a message or some information is to be broadcast to a set of processes.



Many-to-many relationship



- ▶ Many senders send to a mailbox.
- ▶ Many receivers receive from the mailbox.
- ▶ A many-to-many relationship allows multiple server processes to provide concurrent service to multiple clients.



Process-mailbox association

- ▶ The **association of processes** to mailboxes can be either **static or dynamic**.
- ▶ A **one-to-one relationship** is typically defined **d statically** and **permanently**.
- ▶ **Ports are often statically associated with a process:** that is, the port is created and assigned to the process permanently.
- ▶ When there are **many senders**, the **association** of a sender to a mailbox may occur **dynamically**.
- ▶ Primitives such as **connect** and **disconnect** may be used for this purpose.



Mailbox ownership

- ▶ A port is typically created and owned by the receiving process.
- ▶ When the owner process of a mailbox is terminated, the mailbox is also destroyed.
- ▶ Some OS offer a create-mailbox service, which can be called by a process, and in response the OS creates a mailbox. This service has two variations.
 - ▶ The OS may assign mailbox ownership to the process which calls create-mailbox service.
 - ▶ When the owner terminates, the mailbox is deallocated.
 - ▶ The OS may assign mailbox ownership to itself.
 - ▶ To destroy a mailbox, a process must request the OS explicitly.



Message format

- ▶ The **format of the message** depends on:
 - ▶ The **objectives of the messaging facility**, and
 - ▶ Whether the facility runs **on a single computer or on a distributed system.**
- ▶ **Some operating system designers prefer short, fixed-length messages to minimize processing and storage overhead.**
- ▶ **If a large amount of data is to be passed, the data can be placed in a file and the message carries the file handle.**
- ▶ **A more flexible approach is to allow variable-length messages.**



type = TEXT
type = IMAGE
type = REQUEST
type = RESPONSE
type = ERROR

Variable-length message format

- ▶ The header may contain a **type** field to discriminate among various types of messages.
- ▶ There may also be additional control information, such as:
 - ▶ Pointer field to create a linked list of messages,
 - ▶ Sequence number to keep track of order of messages,
 - ▶ Priority field.

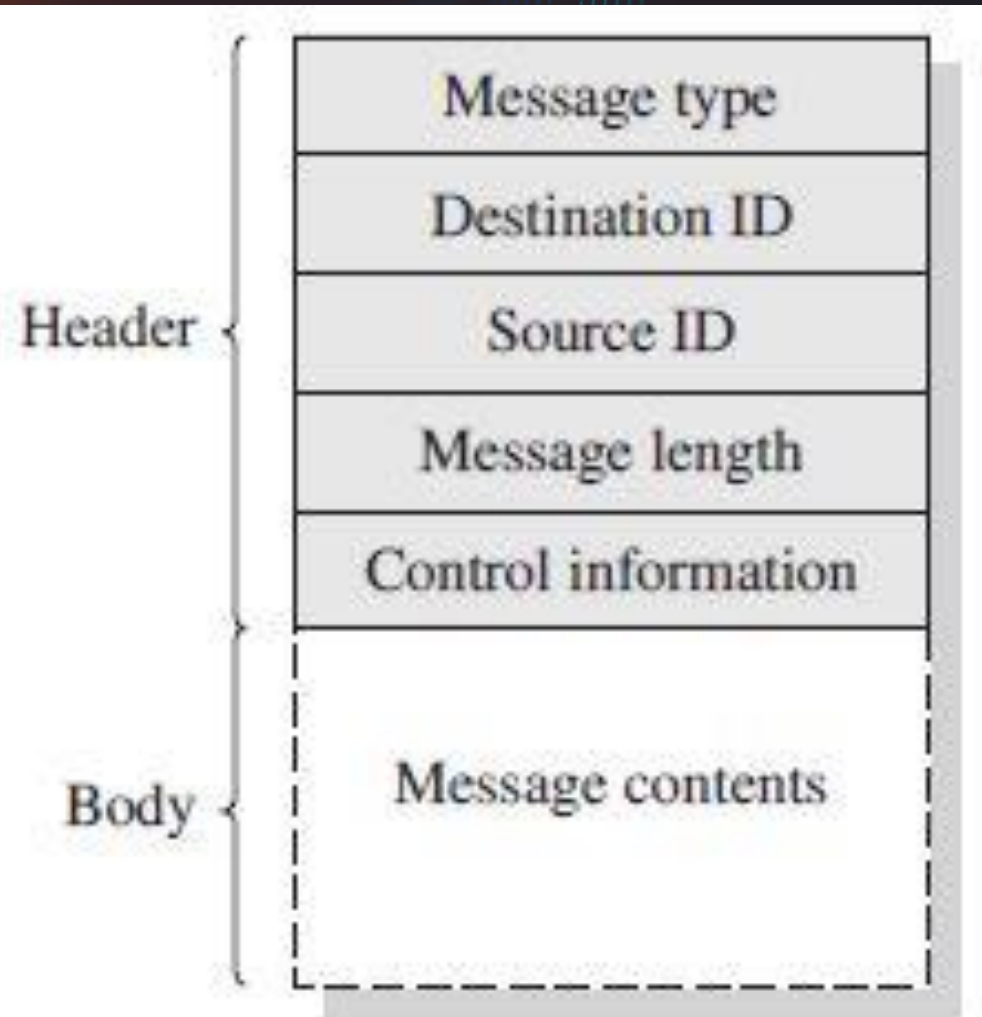


Figure 5.19 General Message Format



Queueing discipline

- ▶ The simplest queueing discipline is **first-in-first-out**.
- ▶ An alternative is to allow the specifying of message **priority**.
 - ▶ Different message types may have different priorities.
 - ▶ The sender may designate priority levels to outgoing messages.
- ▶ Another alternative is to allow the receiver to inspect the message queue and select which message to receive next.




```

/* program mutualexclusion */
const int n = /* number of process */
void P(int i)
{
    message msg;
    while (true) {
        receive (box, msg);
        /* critical section */;
        send (box, msg);
        /* remainder */;
    }
}

```

assuming blocking receive and non blocking send

this code makes sure that if a message is sent one process reads it and the rest are blocked

if message queue is empty all processes are blocked and when message is received only one process is activated and given the message

```

void main()
{
    create mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), ..., P(n));
}

```

if message, receive it else the process blocks

process placed message ack in the mail box

shared mailbox

send an empty message null in the box

n process creates in parallel all resources me

Figure 5.20 Mutual Exclusion Using Messages



- ▶ The main program creates a mailbox.
- ▶ It places a **null** message in the box.
- ▶ It starts three processes in parallel, all of which issue blocking receive.



- ▶ Assume P1 to be the fastest.
- ▶ P1 receives the null message. All others are blocked.
- ▶ P1 enters critical section.



- ▶ When P1 leaves critical section, it send a null message to box.
- ▶ P3 receives the null message. P2 is still blocked.
- ▶ P3 enters critical section.



- ▶ P1 loops back and issues blocking receive.
- ▶ When P3 leaves critical section, it send a null message to box.
- ▶ P2 receives the null message. P1 is still blocked.
- ▶ P2 enters critical section.



```
const int
    capacity = /* buffering capacity */ ;
    null = /* empty message */ ;
int i;
```

size of the buffer
empty message

```
void producer()
{
    message pmsg;
    while (true) {
        receive (mayproduce, pmsg);
        pmsg = produce();
        send (mayconsume, pmsg);
    }
}
```

this process remains blocked until message is received

```
void consumer()
{
    message cmsg;
    while (true) {
        receive (mayconsume, cmsg);
        consume (cmsg);
        send (mayproduce, null);
    }
}
```

the blocking rec, rec a msg from myproduce mailbox and store it in a variable pmsg
the item for buffer is produced and kept in pmsg
non blocking send is used to send the pmsg(message)
mayconsume is a buffer

```
void main()
{
    two mailboxes are used
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++) send (mayproduce, null);
    parbegin (producer, consumer);
}
```

blocking rec
if message (cmsg) received consume the item
non blocking send to mayProd
even if there is one message in mayProduce, producer will execute since waiting for a message

mayproduce mailbox is filled
with all the null messages

Figure 5.21 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages

Classical Synchronization Problems

THE READERS / WRITERS PROBLEM (RWP)



Problem statement

- ▶ There is a data area shared among a number of processes.
- ▶ The data area could be a file, a block of main memory, or even a bank of processor registers.
- ▶ There are a number of processes that only read the data area (readers) and a number that only write to the data area (writers).
- ▶ The following conditions must be satisfied:

no me required 1. Any number of readers may simultaneously read the file.

2. Only one writer at a time may write to the file.

3. If a writer is writing to the file, no reader may read it.

- ▶ Thus, readers are processes that are not required to exclude one another, and writers are processes that are required to exclude all other processes, readers and writers alike.



Difference between RWP and ME

- ▶ In the general problem of enforcing ME over a shared modifiable resource, all processes read and write the critical resource.
- ▶ In the readers / writers problem, readers do not also write to the data area, nor do writers read the data area while writing.
- ▶ If the general solution of enforcing ME is used for the readers / writers problem, we will be serializing not only the accesses of the writers, but also those of the readers.
- ▶ Readers do not modify the shared resource, so implementing ME for them will make the system intolerably slow.
- ▶ Instead, we must focus on:
 - ▶ Preventing writers from interfering with each other, and
 - ▶ Preventing reading while writing is in progress.



Difference between RWP and PCP

- ▶ Can the producer / consumer problem be considered a special case of the readers / writers problem with a single writer (the producer) and a single reader (the consumer)?
because both prod and consumer are readers writers both

▶ The answer is no.

- ▶ The producer is not just a writer. It must read queue pointers to determine where to write the next item, and it must determine if the buffer is full.
- ▶ Similarly, the consumer is not just a reader, because it must adjust the queue pointers to show that it has removed a unit from the buffer.



Resources

- ▶ Anything that a process may ask for is a **resource**.
- ▶ There are two types of resources:
 - ▶ Reusable resources,
 - ▶ Consumable resources.
- ▶ A **reusable resource** can be safely used by one process at a time and it is not depleted by this use.
- ▶ A **consumable resource** is one that can be created and destroyed. When a process uses a consumable resource, it is depleted by this use.



Point to note

The producer / consumer problem involves consumable resources.



Solutions to readers / writers problem



- ▶ There are two solutions to the readers / writers problem:
 - ▶ Readers have priority
 - ▶ Writers have priority
- ▶ Readers have priority: If many readers and a writer arrive together, the readers are allowed access to the resource first.
- ▶ Writers have priority: If many readers and a writer arrive together, the writer is allowed access to the resource first.



```
/* program readersandwriters */
int readcount; tracks readers
semaphore x = 1, wsem = 1;
```

the solution does not change
for multiple reader and writers

x is semaphore for readcount

```
void reader()
```

```
{
    while (true){
        semWait (x); first reader enters and not blocked
        readcount++;
        if(readcount == 1)
            semWait (wsem); will be blocked if writer exists otherwise will run, also will block writers
        semSignal (x); since no writer any reader can come, x is for me on readers so that no two
        READUNIT(); readers update it together
        semWait (x);
        readcount--;
        if(readcount == 0)
            semSignal (wsem); last reader signals the writer that all readers have read the value
        semSignal (x);
    }
}
```

Critical section

```
void writer()
```

```
{
    while (true){
        semWait (wsem); as long as one writer in cs no one allowed, will be blocked since wsem = 0
        WRITEUNIT(); when writer
        semSignal (wsem);
    }
}
```

Critical section

```
void main()
```

```
{
    readcount = 0; 0 readers are accessing the file initially
    parbegin (reader, writer); concurrent processes
}
```

Figure 5.22 A Solution to the Readers/Writers Problem Using Semaphore: Readers Have Priority



```

/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true){
        semWait (x);
        readcount++;
        if(readcount == 1)
            semWait (wsem);
        semSignal (x);
        READUNIT();
        semWait (x);
        readcount--;
        if(readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true){
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}
void main()
{
    readcount = 0;
    parbegin (reader,writer);
}

```

	readcount	x	wsem	Process	Action
1	0	1, Empty	1, Empty		
2			0, Empty	Writer 1	semWait(wsem)
3					WRITEUNIT
4		0, Empty		Reader 1	semWait(x)
5	1				readcount++
6		-1, R2		Reader 2	semWait(x)
7			-1, R1	Reader 1	semWait(wsem)
8		-2, R2, R3		Reader 3	semWait(x)
9			-2, R1, W2	Writer 2	semWait(wsem)
10			-1, W2	Writer 1	semSignal(wsem)
11		-1, R3		Reader 1	semSignal(x)
12					READUNIT
13	2			Reader 2	readcount++
14		0, Empty			semSignal(x)

Figure 5.22 A Solution to the Reader Priority


```

/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true){
        semWait (x);
        readcount++;
        if(readcount == 1)
            semWait (wsem);
        semSignal (x);
        READUNIT();
        semWait (x);
        readcount--;
        if(readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true){
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}
void main()
{
    readcount = 0;
    parbegin (reader,writer);
}

```

	readcount	x	wsem	Process	Action
15	2	0, Empty	-1, W2	Reader 2	READUNIT
16	3			Reader 3	readcount++
17		1, Empty			semSignal(x)
18					READUNIT
19		0, Empty		Reader 1	semWait(x)
20	2				readcount--
21		-1, R2		Reader 2	semWait(x)
22		-2, R2, R3		Reader 3	semWait(x)
23		-1, R3		Reader 1	semSignal(x)
24	1			Reader 2	readcount--
25		0, Empty			semSignal(x)
26	0			Reader 3	readcount--
27			0, Empty		semSignal(wsem)
28		1, Empty			semSignal(x)

Figure 5.22 A Solution to the Reader Priority

Lessons learned

- ▶ We explored interprocess communication using message passing.
- ▶ We learned how to use messages to enforce mutual exclusion.
- ▶ We learned how to use messages to solve the producer / consumer problem.
- ▶ We explored the readers / writers problem.



Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.

